COMPILER CONSTRUCTION LAB · WS 2026/27 · PA1

A ChocoPy front-end: lexing and parsing

JFlex scanner · CUP LALR(1) parser · JSON AST with error recovery

Group 4

Omar Kamel · Malak Chaouch

University of Passau
Chair of Software Engineering I

OVERVIEW · OMAR

Four problems that took most of our time

1

INDENT and DEDENT

several tokens from one line

2

Exact locations

every node as in the reference

3

Error recovery

keep parsing after an error

4

Knowing it is right

--test skips end positions

35 / 35

sample and student tests pass

11,329

programs parsed exactly like the reference

2

known problems we left in, at the end

Indentation: one rule, a stack and yypushback

```
{WhiteSpace}* [^ \t\r\n#] {
  yypushback(1);          // keep the first char
  int indentation = indentationOf(yytext());
  if (indentation > indentStack.peek()) {
    indentStack.push(indentation);
    yybegin(LINE_BODY);
    return INDENT;
  }
  if (indentation < indentStack.peek()) {
    indentStack.pop();
    yypushback(width);    // fire again
    return DEDENT;
  }
  yybegin(LINE_BODY);     // same level
}
```

Deeper

push the level, one INDENT

Same level

no token, continue the line

Shallower

pop, one DEDENT, push the whitespace back: the rule fires again until the level matches

Tabs advance to the next multiple of 8. At the end of input: NEWLINE if missing, one DEDENT per open level, EOF. **ChocoPy.jflex**, lines 162–192 and 283–298

Example: the tokens of two nested blocks

Line	Source	Stack	Tokens
1	if x:	[0]	IF IDENTIFIER COLON NEWLINE
2	····if y:	[0, 4]	**INDENT** IF IDENTIFIER COLON NEWLINE
3	········a = 1	[0, 4, 8]	**INDENT** IDENTIFIER EQ INTEGER NEWLINE
4	(blank line)	[0, 4, 8]	none
5	····# done	[0, 4, 8]	none: comment-only lines are skipped
6	b = 2	[0]	**DEDENT DEDENT** IDENTIFIER EQ INTEGER NEWLINE
end		[0]	EOF

Line 6 runs the indentation rule three times: two DEDENTs, then the same level. The comment on line 5 is indented by 4, but it neither closes a block nor breaks it.

LEXER · OMAR

Literals and the edge cases we checked

Input	Token	Why
"abc"	IDSTRING	looks like an identifier, so it may name a type
"a b"	STRING	any other printable ASCII text
"He\"llo"	STRING He"llo	escapes \" \n \t \\ are applied to the value
"Hell\o"	UNRECOGNIZED	illegal escape: the whole literal is one bad token
2147483648	UNRECOGNIZED	larger than 2^31 – 1
007	INTEGER ×3	0, 0, 7: no leading zeros, maximal munch
\r, \n, \r\n	NEWLINE	all three line ends, on every platform

Every row was compared with the reference lexer. Fuzzing found one crash here: a 20-digit literal overflowed **long**; now any number that does not fit an **int** is UNRECOGNIZED.

Precedence without a single conflict

```
/* lowest precedence first */
or_expr    ::= or_expr OR and_expr
and_expr   ::= and_expr AND not_expr
not_expr   ::= NOT not_expr | comparison
comparison ::= arith LT arith | …
arith      ::= arith PLUS term | …
term       ::= term TIMES factor | …
factor     ::= MINUS factor | atom
```

names shortened; ChocoPy.cup lines 311–354

Each level only uses itself and the next one, so the grammar is not ambiguous: 0 conflicts even with all precedence lines deleted.

> **`a < b < c`**
>
> error at the second <, as in the reference

> **`True == not False`**
>
> error at not: it sits above comparison

PARSER · MALAK

Building the AST: elif and exact locations

```
if a:
    x = 1
elif b:
    x = 2
else:
    x = 3
```

IfStmt condition a, thenBody [x = 1], elseBody:

> **IfStmt** condition b, thenBody [x = 2], elseBody [x = 3]
>
> starts at the elif keyword, ends at the DEDENT of the else block

Leftmost to rightmost

A node spans its CUP symbols: (1 + 2) * 3 covers columns 1–11, parentheses included.

Statements stop early

A statement node ends before its NEWLINE, but a function ends where that NEWLINE ends.

Blocks end at DEDENT

if, while and for end at the DEDENT, which spans the indentation of the next line.

PARSER · MALAK

Where does a Program start and end?

`pass`

no statement in the AST, but the Program is [1, 1, 1, 5]

`# c`
`x: int = 1`

starts at x, not at the comment: [2, 1, 2, 10]

empty file

[0, 0, 0, 0]

```
// every token passes through here
public Symbol scan() {
  Symbol t = super.scan();
  if (start == null && t.sym != NEWLINE)
    start = t.xleft;
  if (t.sym == NEWLINE || t.sym == DEDENT)
    end = t.xright;
  return t;
}
```

The lists of declarations and statements cannot give this location, so the parser records it while the tokens arrive. ChocoPy.cup lines 46–61 and 149–163.

A broken line costs only that line

```
x = 1 +          ← 1:8 NEWLINE
y = 2            kept
```

```
class A(object):
    x: int = 1     kept
    y = 2          ← 3:7 EQ
    z: int = 3     kept
```

Without an error rule, CUP stops at the first error. With one, it skips to the end of the line and goes on.

```
statement ::= …
  | error NEWLINE

class_decl_list ::= …
  | class_decl_list
    error NEWLINE
```

Both files: same errors and same AST as the reference.

A broken header: still check the body

```
def f(a: int, b) -> int:   ← 1:16 RPAREN
    return a +             ← 2:15 NEWLINE
print(1)                   kept
```

The problem

Skipping only the header line leaves an indented body that fits nowhere, so the error inside it would be lost.

```
function_definition ::= …
  | DEF error NEWLINE
    INDENT function_body
    DEDENT

declaration ::= …
  | CLASS error NEWLINE
    INDENT class_decl_list
    DEDENT
```

The body is parsed as usual and then dropped. Both errors match the reference.

ERROR RECOVERY · MALAK

Result on our bad.py

9

errors from our parser, each one also reported by the reference

17

errors from the reference parser, with follow-ups at DEDENTs

```
print(1 + )                          ← 3:11 RPAREN
print(2)                             kept
…
value: int = 1 + 2                   ← 11:6 COLON
def broken(a: int, b) -> int:
    …                                ← 18:1 DEDENT
class Pair(object):
    …
    third: int = 2                   ← 24:10 COLON
if 1 < value < 3:                    ← 28:14 LT
value = value && 1                   ← 33:15 UNRECOGNIZED
print(True == not False)             ← 34:15 NOT
while value > 0                      ← 38:16 NEWLINE
```

TESTING · OMAR

The reference parser as a test oracle

35 sample and student tests

99 hand-written edge cases

11,195 random mutants

Both parsers in one JVM

valid program: the whole JSON must be equal, locations included

invalid program: the first error at the same line and column

11,329

identical, 0 failures

Mutants

delete, insert, duplicate or swap tokens, add or remove indentation in every test file

Bugs it found in our code

a crash on huge integers, DEDENT columns at EOF, the start of a program that begins with pass

TESTING · OMAR

What we found in the reference and the material

Where	Finding
Reference lexer	No NEWLINE at end of input without a final line break; hidden because the driver appends "\n"
Reference messages	"Parse error near token DEDENT: x" shows the text of the previous token
Local --test mode	Checks where nodes start but not where they end, so wrong end positions still pass
Specification	Kind NonlocalDecl is NonLocalDecl in the AST and outputs; typo "stater code"
Manual 2.8.6	"outputs outputs its printed form"

Each item has a small input and the expected output in our README.

NOT FIXED · OMAR

Two problems we left in, and why

Fewer follow-up errors

bad.py: **9** errors from us, **17** from the reference.

Why: after recovering, CUP reports the next error only once 3 tokens parse (error_sync_size).

Left in: only the first error is graded, and our harness checks only that one too.

Positions outside the file

The reference puts DEDENTs at the end of input on a line after the file: `[1, 1, 5, 3]` for 4 lines.

Left in: we copy it on purpose, because the hidden tests compare with the reference.

Reported as a bug in our README.

SUMMARY

Who did what

Omar Kamel

LEXER AND TESTING

ChocoPy.jflex: line structure, INDENT and DEDENT, tabs, string and integer literals, reserved keywords, UNRECOGNIZED; good.py; differential testing against the reference

Malak Chaouch

GRAMMAR, AST AND RECOVERY

ChocoPy.cup: declarations, statements, expressions and precedences, elif desugaring, exact node locations, error rules; bad.py

We wrote the README together and reviewed every part of each other's code, so both of us can explain the whole project. About 30 hours in total.

FOR THE Q&A

Where each part lives in the code

Part	File	Lines
Indentation stack and the tab rule	ChocoPy.jflex	83-103
Start of a line: INDENT and DEDENT	ChocoPy.jflex	162-192
Keywords, literals, operators	ChocoPy.jflex	198-279
End of input	ChocoPy.jflex	283-298
Program, declarations and classes	ChocoPy.cup	149-200
Functions and their bodies	ChocoPy.cup	202-235
Statements, blocks and elif	ChocoPy.cup	255-309
Expressions, one rule per level	ChocoPy.cup	311-392

PA1 · GROUP 4

Thank you. Questions?

Lexer, grammar, error recovery, testing: ask any of us.

```
mvn clean package
java -cp "chocopy-ref.jar;target/assignment.jar" chocopy.ChocoPy --pass=s --dir src/test/data/pa1 --test
```